

CHANDIGARH UNIVERSITY

UNIVERSITY INSTITUTE OF COMPUTING · DEPARTMENT OF COMPUTER APPLICATIONS

FINAL-YEAR MAJOR PROJECT REPORT

Realtime Chat

A Cross-Platform Realtime Messaging Application built with Expo & Supabase

Submitted by	Ishika Sai
University ID (UID)	023BCA110006
Programme	BCA — VI Semester (6th) · BCA-V2
Batch	2023 – 2026
Mentor / Supervisor	Sahil Jindal

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Computer Applications (BCA) · Academic Session 2025–2026
Date of Submission: 01 July 2026

Certificate

This is to certify that the project report entitled “**Realtime Chat — A Cross-Platform Realtime Messaging Application**” is a bona-fide record of the project work carried out by **Ishika Sai** (UID: **023BCA110006**), a student of the **Bachelor of Computer Applications (BCA-V2)** programme, Chandigarh University, during the academic session 2025–2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications.

The work presented in this report is original and has been carried out under my supervision and guidance. To the best of my knowledge, the matter embodied in this report has not been submitted to any other university or institution for the award of any other degree or diploma.

Mr. Sahil Jindal

Project Mentor / Supervisor
Department of Computer Applications
Chandigarh University

Head of Department

Department of Computer Applications
University Institute of Computing
Chandigarh University

Date: 01 July 2026 **Place:** Gharuan, Mohali (Punjab)

Declaration

I, **Ishika Sai**, bearing University ID **023BCA110006**, hereby declare that the project work presented in this report entitled “**Realtime Chat — A Cross-Platform Realtime Messaging Application**” is an authentic record of my own work carried out under the supervision of **Mr. Sahil Jindal** as part of the requirements for the degree of Bachelor of Computer Applications (BCA-V2) at Chandigarh University.

I further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma in this or any other institute or university. All sources of information and assistance used in the preparation of this report have been duly acknowledged, and all external references and libraries used have been appropriately cited.

Date: 01 July 2026

Ishika Sai

UID: 023BCA110006

BCA-V2, 2023–2026

Chandigarh University

Acknowledgement

The successful completion of any task is incomplete without acknowledging the people who made it possible. I take this opportunity to express my sincere gratitude to all those who supported and guided me throughout the course of this project.

First and foremost, I would like to express my deepest gratitude to my project mentor, **Mr. Sahil Jindal**, Department of Computer Applications, Chandigarh University, for his invaluable guidance, constant encouragement, and constructive feedback at every stage of this project. His insights into system design, security, and modern application development shaped both the project and this report in significant ways.

I am thankful to the **Head of the Department** and the entire faculty of the University Institute of Computing for providing a stimulating academic environment and the resources necessary to undertake this work. I also extend my appreciation to the open-source community behind **Expo**, **React Native**, and **Supabase**, whose documentation and tools formed the technical backbone of this application.

Finally, I owe my heartfelt thanks to my family and friends for their patience, motivation, and unwavering support throughout my academic journey. This project stands as a testament to their constant encouragement.

Ishika Sai

UID: 023BCA110006

Abstract

Realtime Chat is a cross-platform instant-messaging application for Android that delivers a modern, WhatsApp-style communication experience while being built entirely without a traditional, self-managed backend server. The application is implemented using **Expo (React Native, SDK 52)** on the client side and the **Supabase** Backend-as-a-Service platform — comprising a managed PostgreSQL database, a Realtime engine, authentication, object storage, and serverless Edge Functions — on the server side. This architecture demonstrates how a feature-rich, secure, and scalable messaging product can be assembled rapidly using managed cloud primitives rather than bespoke infrastructure.

The system supports one-to-one direct messages and group channels, real-time delivery of new, edited, and deleted messages, emoji reactions, read receipts, unread-message badges, online-presence indicators, typing indicators, and in-chat image sharing. A central contribution of the project is a **push-notification pipeline** that delivers alerts to users even when the application is completely closed: a PostgreSQL trigger built on the `pg_net` extension invokes a Supabase Edge Function on every new message, which in turn dispatches notifications through the Expo Push service to **Firebase Cloud Messaging (FCM)** and on to the recipient's device.

Security is enforced at the database layer through comprehensive **Row Level Security (RLS)** policies, made recursion-safe through a `SECURITY DEFINER` helper function, ensuring that users can only ever read and write the conversations of which they are members. The application is packaged into an installable Android APK using Expo's EAS Build cloud pipeline. This report documents the motivation, requirements analysis, system design, implementation details, testing strategy, and deployment of the application, and concludes with an evaluation of the results and directions for future enhancement.

Keywords:

Realtime Chat

React Native

Expo

Supabase

PostgreSQL

Row Level Security

Push Notifications

FCM

Edge Functions

WebSockets

Table of Contents

	Certificate · Declaration · Acknowledgement · Abstract	i–iv
Ch 1	Introduction	1
	1.1 Overview 1.2 Problem Statement 1.3 Motivation 1.4 Objectives 1.5 Scope 1.6 Organization	
Ch 2	Literature Review & Existing Systems	6
Ch 3	System Analysis & Requirements	11
Ch 4	System Design	16
Ch 5	Implementation	25
Ch 6	Testing & Evaluation	35
Ch 7	Deployment	40
Ch 8	Results & Screenshots	43
Ch 9	Conclusion & Future Scope	45
	References	48
	Appendix A — Source Listings & Database Schema	49

List of Figures

Fig 4.1	High-Level System Architecture	17
Fig 4.2	Level-0 Data Flow Diagram (Context Diagram)	19
Fig 4.3	Level-1 Data Flow Diagram	19
Fig 4.4	Entity-Relationship Diagram	21
Fig 4.5	Sequence Diagram — Push Notification Pipeline	23

List of Tables

Tbl 2.1	Comparison of Existing Messaging Applications	8
Tbl 2.2	Comparison of Real-Time Communication Approaches	10
Tbl 3.1	Functional Requirements	12
Tbl 6.1	Test Cases & Results	37



Communication is the single most-used category of software on a modern smartphone. This chapter introduces the Realtime Chat application, the problem it addresses, the motivation behind building it on a serverless backend, the objectives it sets out to achieve, and the boundaries of its scope.

1.1 Overview

Instant messaging has become the default mode of personal and professional communication. Applications such as WhatsApp, Telegram, and Signal collectively serve billions of users, and the expectations they have established — instantaneous delivery, presence awareness, media sharing, and notifications that reach the user even when the application is closed — now define the baseline for any new entrant in the space. Building such a system traditionally required substantial backend engineering: persistent WebSocket servers, a horizontally scalable database, a message queue, an authentication service, a media store, and a notification gateway, each demanding configuration, monitoring, and maintenance.

Realtime Chat is a mobile application that delivers this full experience while deliberately avoiding a hand-written backend. It is built with **Expo (React Native, SDK 52)** for the Android client and uses the **Supabase** Backend-as-a-Service platform for every server-side concern. Supabase provides a managed PostgreSQL database, a Realtime engine that streams database changes over WebSockets, an authentication service, object storage, and serverless Edge Functions. By composing these managed primitives, the entire backend is expressed as database schema, security policies, and a single small serverless function — there is no server process for the developer to operate.

The result is a fully featured messenger that supports private one-to-one conversations and group channels, real-time message delivery and editing, emoji reactions, read receipts, online presence, typing indicators, image sharing, and — crucially — push notifications that are delivered through Firebase Cloud Messaging even when the application has been swiped away.

1.2 Problem Statement

Despite the ubiquity of messaging applications, building one remains a non-trivial software-engineering exercise, particularly for an individual developer or a student team. The core difficulties are:

- **Real-time delivery.** Messages must appear on the recipient's screen within milliseconds, which rules out naïve request-response polling and demands a persistent bidirectional transport.

- **State synchronization.** Presence (who is online), typing indicators, read receipts, and reaction counts must be kept consistent across multiple devices simultaneously.
- **Security and privacy.** A conversation between two users must be invisible to everyone else. Enforcing this correctly across every read and write path is error-prone when implemented in application code.
- **Offline notification.** The hardest requirement is notifying a user whose application is not running, which requires integration with the operating system’s push infrastructure.
- **Operational burden.** Each of the above traditionally implies servers to provision, scale, secure, and pay for around the clock.

The problem this project addresses is therefore twofold: to deliver a complete, correct, and secure messaging experience, and to do so using an architecture that an individual can build and operate without managing any server infrastructure.

PROBLEM, IN ONE SENTENCE

How can a full-featured, secure, real-time messaging application — including offline push notifications — be designed and implemented without writing or operating a custom backend server?

1.3 Motivation

The motivation for this project arises from the convergence of two trends. First, the rise of **Backend-as-a-Service** platforms has dramatically lowered the barrier to building data-intensive applications; capabilities that once required dedicated teams are now available as managed services with generous free tiers. Second, **React Native** and **Expo** have matured to the point where a single JavaScript codebase can produce a polished, native-feeling mobile application with access to device capabilities such as the camera, photo library, and push notifications.

Exploring whether these two trends can be combined to reproduce a “WhatsApp-class” feature set is both an instructive academic exercise and a practically valuable one. It forces a deep engagement with several important computer-science concepts — relational data modeling, the publish-subscribe pattern, database-level access control, event-driven serverless computing, and mobile notification architecture — all within a single coherent project. The motivation is thus to learn these concepts not in isolation but as they interlock in a real product, and to demonstrate that modern managed infrastructure makes ambitious applications accessible to students and independent developers.

1.4 Objectives

The project is guided by the following concrete objectives:

1. To design a normalized relational data model capable of representing users, direct messages, group channels, membership, messages, reactions, and notification tokens.

2. To implement instantaneous, bidirectional message delivery using a publish-subscribe real-time transport rather than polling.
3. To support a rich messaging feature set: editing and deletion of messages, emoji reactions, read receipts, unread-count badges, online presence, and typing indicators.
4. To enable in-chat image sharing backed by secure, access-controlled object storage.
5. To enforce conversation privacy at the database layer through Row Level Security, guaranteeing isolation independently of client-side code.
6. To build a notification pipeline that reaches users even when the application is terminated, using a database trigger, a serverless function, and Firebase Cloud Messaging.
7. To package and distribute the application as an installable Android APK through an automated cloud build pipeline.

1.5 Scope of the Project

The scope of the present work is bounded as follows. The application targets the **Android** platform; while the React Native codebase is portable to iOS, Apple-specific distribution and push credentials are out of scope. The authentication model is intentionally lightweight — users identify themselves with a chosen username via anonymous authentication — which keeps onboarding frictionless and focuses the project on messaging mechanics rather than credential management. The feature set covers text and image messages; audio and video messages, voice and video calling, and end-to-end encryption are explicitly deferred to future work and are discussed in Chapter 9.

Within these bounds, the project delivers a complete, working product: a published Android application backed by a live cloud database, with all the real-time and notification features described above functioning end to end.

1.6 Organization of the Report

Chapter 2

Reviews existing messaging applications and compares the available real-time communication techniques.

Chapter 3

Presents the requirements analysis, feasibility study, and use-case descriptions.

Chapter 4

Details the system design — architecture, data-flow diagrams, the entity-relationship model, and the security design.

Chapter 5

Describes the implementation module by module, with representative code.

Chapter 6

Covers the testing strategy, test cases, and evaluation of results.

Chapters 7–9

Address deployment, results and screenshots, and conclusions with future scope, followed by references and an appendix.

2

CHAPTER TWO

Literature Review & Existing Systems

Before designing a new system it is essential to study the landscape of existing solutions and the techniques that underpin them. This chapter surveys leading messaging applications, examines the architectural patterns they employ, and compares the principal approaches to real-time communication on which any such system must be founded.

2.1 Survey of Existing Messaging Applications

Four applications dominate contemporary discussion of mobile messaging, each representing a distinct point in the design space.

2.1.1 WhatsApp

WhatsApp is the most widely used messaging application in the world. Architecturally it is notable for its use of the **XMPP**-derived signalling protocol over persistent connections, a backend historically built on **Erlang** for massive concurrency, and end-to-end encryption based on the Signal Protocol. Its feature set — delivery and read receipts (the familiar single, double, and blue ticks), presence (“last seen” and “online”), typing indicators, media sharing, and reliable background notifications — has effectively defined user expectations for the entire category. WhatsApp demonstrates that a messaging product lives or dies by the reliability of its real-time delivery and its notifications.

2.1.2 Telegram

Telegram differentiates itself through a cloud-first model in which message history is stored on the server and synchronized across an unlimited number of devices. It uses a custom protocol, **MTPROTO**, and emphasizes large group channels, bots, and a rich client API. Telegram illustrates the value of treating the server as the single source of truth for conversation state — an approach this project adopts by storing all messages in a central PostgreSQL database.

2.1.3 Signal

Signal is the reference implementation of privacy-first messaging; the Signal Protocol it pioneered is the basis of end-to-end encryption in WhatsApp and others. Signal deliberately minimizes the metadata it retains. While end-to-end encryption is beyond the scope of the present project, Signal’s design underscores the importance of a rigorous security model — a concern this project addresses at the database layer through Row Level Security.

2.1.4 Discord & Slack

Discord and Slack represent the “channel-centric” branch of messaging, organizing conversation into persistent topic-based rooms rather than purely person-to-person threads. Both rely heavily on the WebSocket protocol for real-time updates and on presence systems to show member availability. Their model directly informs this project’s support for both **direct messages** and **channels** within a single unified data model.

2.2 Comparative Analysis

Table 2.1 situates the proposed Realtime Chat application against these established systems across the features most relevant to this project. The intent is not to claim parity with products built by large engineering organizations, but to clarify which capabilities are in scope and how they are realized.

Capability	WhatsApp	Telegram	Discord	Realtime Chat (Proposed)
Direct messages	Yes	Yes	Yes	Yes
Group channels	Yes	Yes	Yes	Yes
Real-time delivery	XMPP/custom	MTPROTO	WebSocket	Supabase Realtime (WebSocket)
Presence / online status	Yes	Yes	Yes	Yes (Presence channel)
Typing indicators	Yes	Yes	Yes	Yes (Broadcast)
Read receipts	Yes	Yes	Partial	Yes (per-member last-read)
Reactions	Yes	Yes	Yes	Yes
Edit / delete	Yes	Yes	Yes	Yes
Image sharing	Yes	Yes	Yes	Yes (private bucket + signed URL)
Offline push (app closed)	Yes	Yes	Yes	Yes (Edge Function → FCM)
Self-managed backend	Yes (large)	Yes (large)	Yes (large)	None — fully managed (BaaS)

Table 2.1 — Comparison of existing messaging applications with the proposed system.

THE GAP ADDRESSED

Existing systems achieve their feature set with substantial, self-operated backend infrastructure. The proposed system targets the same user-facing capabilities using only managed cloud primitives, demonstrating a viable architecture for individuals and small teams.

2.3 Real-Time Communication Approaches

The defining technical challenge of any messenger is the real-time transport. Several techniques exist, each with different trade-offs in latency, server load, and complexity.

2.3.1 Short Polling

In short polling the client repeatedly asks the server “is there anything new?” at a fixed interval. It is trivial to implement over ordinary HTTP but is fundamentally inefficient: most requests return nothing, latency is bounded below by the polling interval, and server load scales with the number of clients regardless of activity. It is unsuitable for a responsive messenger.

2.3.2 Long Polling

Long polling improves on this by having the server hold each request open until data is available or a timeout elapses. Latency improves, but each client still consumes a connection and the pattern incurs the overhead of repeatedly re-establishing HTTP requests. It was the basis of early web chat but has largely been superseded.

2.3.3 WebSockets

The **WebSocket** protocol establishes a single, persistent, full-duplex TCP connection between client and server, over which either party may send messages at any time with minimal framing overhead. This is the modern standard for real-time applications and provides the low latency and efficiency a messenger requires. The principal cost is operational: maintaining large numbers of stateful, long-lived connections is demanding to scale.

2.3.4 Server-Managed Realtime (the chosen approach)

Supabase Realtime builds on WebSockets but removes the operational burden by managing the connection infrastructure as a service. It exposes three complementary mechanisms: **Postgres Changes**, which streams `INSERT`, `UPDATE`, and `DELETE` events directly from the database to subscribed clients; **Presence**, which tracks the set of clients currently joined to a channel; and **Broadcast**, a low-latency pub-sub channel for ephemeral events such as typing indicators. This project uses all three, obtaining WebSocket-grade responsiveness without operating any WebSocket servers.

Approach	Latency	Efficiency	Implementation Effort	Verdict
Short polling	High ($\geq$ interval)	Poor	Trivial	Rejected
Long polling	Medium	Fair	Moderate	Rejected
Raw WebSockets	Low	Good	High (self-operated)	Viable but costly
Supabase Realtime	Low	Good	Low (managed)	Selected

Table 2.2 — Comparison of real-time communication approaches.

2.4 Summary

The literature confirms that a credible messenger must offer real-time delivery, presence, receipts, media, and reliable offline notification, and that WebSockets are the appropriate transport. The novelty pursued in this project is not in the feature set itself but in the architecture: achieving these features entirely through a managed Backend-as-a-Service, thereby eliminating the operational complexity that has historically made such systems the preserve of large teams. The following chapter translates these observations into a concrete set of requirements.



System Analysis & Requirements

This chapter formalizes what the system must do and the conditions under which it must do it. It enumerates functional and non-functional requirements, assesses feasibility, describes the principal use cases, and states the hardware and software prerequisites.

3.1 Requirements Analysis

Requirements were derived by studying the behaviour expected of established messaging applications (Chapter 2) and distilling the subset appropriate to the project's scope. They are divided into **functional** requirements, which describe the services the system provides, and **non-functional** requirements, which describe the qualities the system must exhibit.

3.2 Functional Requirements

ID	Requirement	Description	Priority
FR-1	User onboarding	A user shall identify themselves by choosing a username and shall be issued a persistent identity via anonymous authentication.	High
FR-2	Direct messaging	A user shall be able to start or resume a one-to-one conversation with any other user.	High
FR-3	Channels	A user shall be able to create, browse, and join group channels.	High
FR-4	Send message	A user shall be able to send text messages that are delivered to recipients in real time.	High
FR-5	Edit / delete	A user shall be able to edit or delete their own messages, with the change reflected for all participants.	Medium
FR-6	Reactions	A user shall be able to attach and remove emoji reactions to any message.	Medium
FR-7	Read receipts	The system shall indicate, in direct messages, whether a sent message has been read.	Medium
FR-8	Unread badges	The conversation list shall display the number of unread messages per room.	Medium
FR-9	Presence	The system shall indicate which participants are currently online.	Medium
FR-10	Typing indicator	The system shall indicate when a participant is composing a message.	Low
FR-11	Image sharing	A user shall be able to attach and send an image from their device's photo library.	Medium
FR-12	User search	A user shall be able to find other users by username.	Medium
FR-13	Push notification	A user shall receive a notification of a new message even when the application is closed.	High
FR-14	Profile	A user shall be able to view and change their username and sign out.	Low

Table 3.1 — Functional requirements of the Realtime Chat application.

3.3 Non-Functional Requirements

Performance

New messages shall appear on recipients' devices within roughly one second under normal network conditions; the UI shall remain responsive while messages stream in.

Security & Privacy

A user shall be able to access only the conversations of which they are a member; isolation shall be enforced at the database layer, not merely in the client.

Scalability

The architecture shall rely on managed services that scale horizontally without developer intervention for the project's expected load.

Reliability

Message persistence shall be guaranteed by the database; notification delivery shall degrade gracefully if a device token is invalid.

Usability

Onboarding shall require no more than choosing a username; the interface shall follow familiar messaging conventions.

Maintainability

The codebase shall be modular, with a clear separation between the data layer, UI components, and screens; schema changes shall be version-controlled as migrations.

Portability

The client shall be written once in React Native so that an iOS build is achievable without rewriting application logic.

Cost-efficiency

The system shall operate within the free tiers of its managed services for the project's scale.

3.4 Feasibility Study

3.4.1 Technical Feasibility

Every capability required by the project is supported by mature, well-documented technology. React Native and Expo provide native device access and a robust UI toolkit; Supabase provides the database, real-time engine, authentication, storage, and serverless compute; and Firebase Cloud Messaging provides the device-level notification gateway. The integration points between these technologies are standard and supported, making the project technically feasible for a single developer.

3.4.2 Economic Feasibility

The project incurs no infrastructure cost at its scale. Supabase's free tier covers the database, real-time connections, storage, and Edge Function invocations required; Firebase Cloud Messaging is free of charge and imposes no message limit; and Expo's EAS Build free tier is sufficient to produce the Android APK. The only optional cost in the wider ecosystem — an Apple Developer subscription — applies solely to iOS distribution, which is out of scope. The project is therefore economically feasible at essentially zero marginal cost.

3.4.3 Operational Feasibility

Because the backend is composed entirely of managed services, there is no server for the developer to provision, patch, or monitor. Routine operation reduces to maintaining the database schema and the single Edge Function. The application installs as a standard Android APK, which any user can side-load. The system is thus operationally feasible without a dedicated operations function.

3.5 Use-Case Descriptions

The principal actors are the **User** (any participant) and two external systems, the **Realtime Service** and the **Push Service**. The salient use cases are described below.

UC-1 · Onboard

Actor: User. The user enters a username; the system creates an anonymous identity and a profile, and joins the user to the #general channel.

UC-2 · Open Conversation

Actor: User. The user selects an existing conversation or starts a new direct message or channel; the system loads message history and subscribes to real-time updates.

UC-3 · Send Message

Actor: User. The user composes and sends a message; it is persisted, streamed to other participants, and triggers a notification.

UC-4 · React / Edit / Delete

Actor: User. The user long-presses a message to add a reaction, or edit/delete their own message; changes propagate in real time.

UC-5 · Share Image

Actor: User. The user attaches an image from the library; it is uploaded to private storage and shared via a signed URL.

UC-6 · Receive Notification

Actor: Push Service. On a new message, the recipient’s device displays a notification even if the app is closed.

3.6 System Requirements

Category	Requirement
Development — Hardware	A development machine with ≥ 8 GB RAM; a physical Android device or emulator for testing.
Development — Software	Node.js 18+; the Expo CLI and EAS CLI; a modern code editor; the Git version-control system; a web browser for the Supabase dashboard.
Backend Services	A Supabase project (PostgreSQL, Realtime, Auth, Storage, Edge Functions); a Firebase project for FCM credentials.
End-User Device	An Android smartphone running Android 7.0 (Nougat) or later, with an internet connection and permission to install applications from outside the Play Store.

3.7 Summary

The analysis establishes a clear, prioritized set of fourteen functional requirements and eight quality attributes, confirms that the project is technically, economically, and operationally feasible, and identifies the actors and use cases that drive the design. These requirements form the contract against which the design of Chapter 4 and the implementation of Chapter 5 are measured.



Design translates the requirements of Chapter 3 into a concrete structure. This chapter presents the architectural design, the flow of data through the system, the relational data model, the real-time channel design, the database-level security model, and the user-interface design.

4.1 Design Methodology & Principles

The system is designed around four guiding principles. First, **the database is the source of truth**: all durable state — messages, membership, reactions, read positions — lives in PostgreSQL, and the client is a view onto it. Second, **security is enforced where the data lives**: access control is expressed as Row Level Security policies on the tables themselves, so no client bug can leak data. Third, **real-time is a first-class concern**: the client subscribes to database changes rather than polling. Fourth, **the server is event-driven and stateless**: the only custom server-side logic is a single Edge Function invoked by a database trigger, with no long-running process to maintain.

4.2 High-Level Architecture

The architecture comprises three tiers: the **mobile client** built with Expo and React Native; the **Supabase backend**, which bundles the database, real-time engine, authentication, storage, and serverless functions; and the **external notification services** (Expo Push and Firebase Cloud Messaging). Figure 4.1 depicts the tiers and the channels of communication between them.

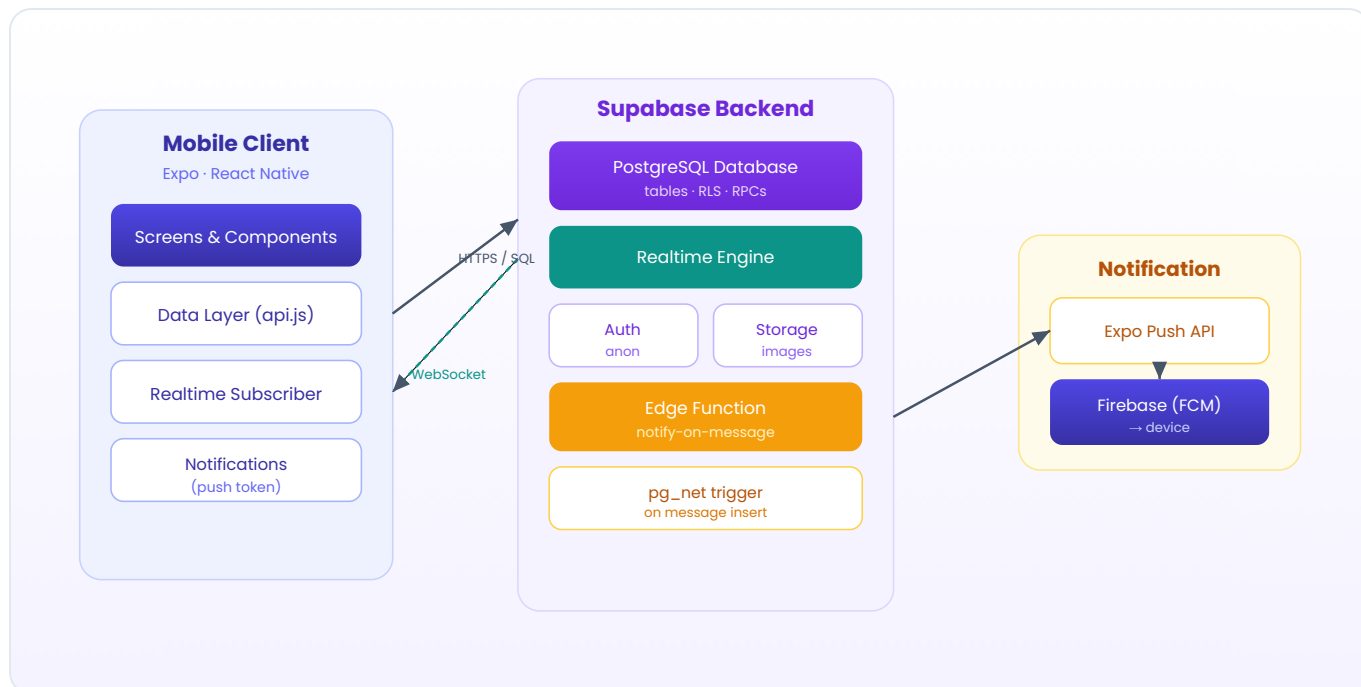


Figure 4.1 — High-level system architecture: the client communicates with Supabase over HTTPS for queries and over a WebSocket for real-time updates; a database trigger drives the notification path to FCM.

The client never holds privileged credentials: it authenticates as an anonymous user and is bound by Row Level Security for every operation. Queries and mutations travel over HTTPS to the auto-generated PostgREST API; real-time updates arrive over a single WebSocket managed by the Realtime engine. The notification path is entirely server-side and is invisible to the sending client.

4.3 Data Flow Diagrams

Data-flow diagrams model the movement of information through the system independently of its implementation. Figure 4.2 is the context (Level-0) diagram, treating the system as a single process interacting with external entities; Figure 4.3 decomposes it into the major internal processes.

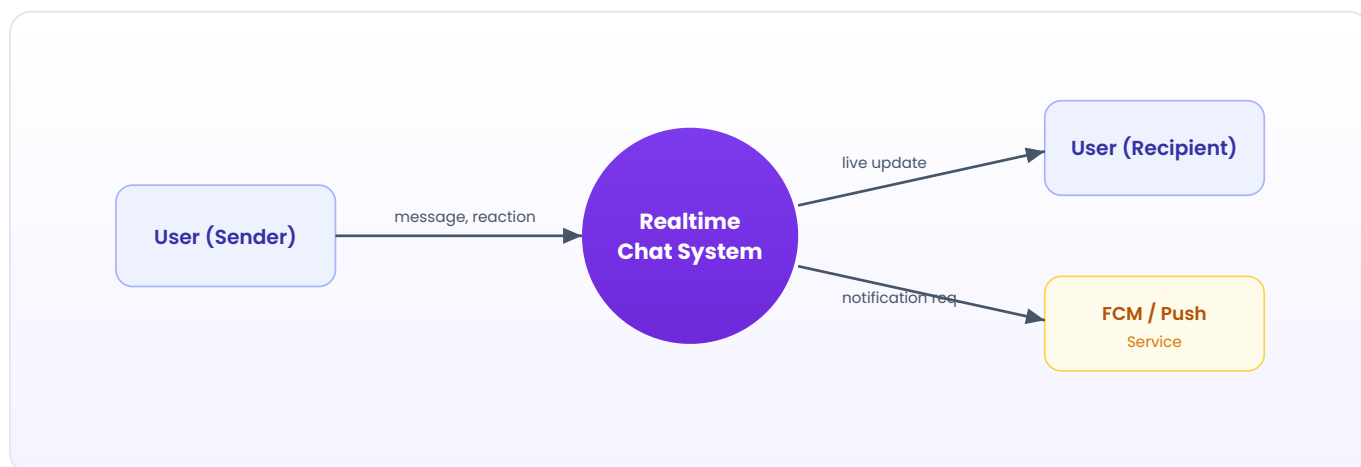


Figure 4.2 — Level-0 (context) data-flow diagram.

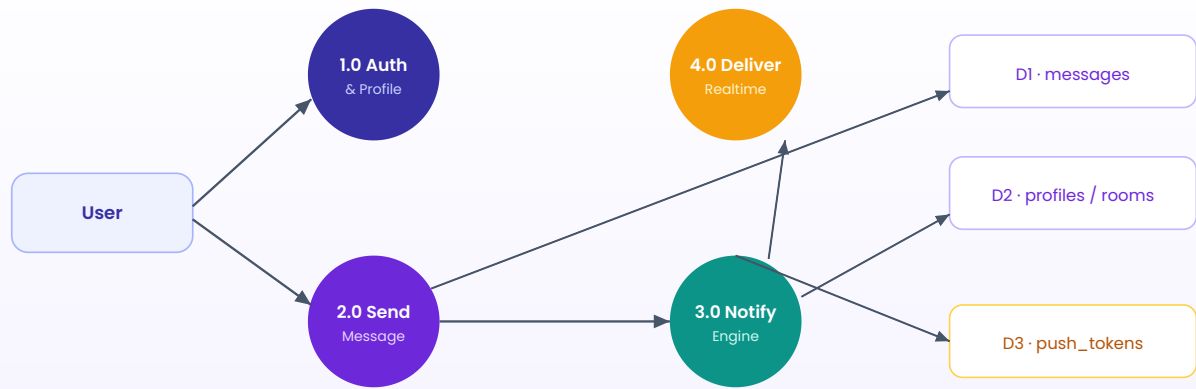


Figure 4.3 — Level-1 data-flow diagram showing authentication, message-send, the notification engine, real-time delivery, and the data stores.

4.4 Database Design

The relational schema is the heart of the system. It is normalized to third normal form and consists of six tables. A single `rooms` table represents both direct messages and channels (distinguished by a type column), which keeps membership, messaging, and real-time subscription logic uniform across the two conversation kinds. Figure 4.4 shows the entity-relationship model.

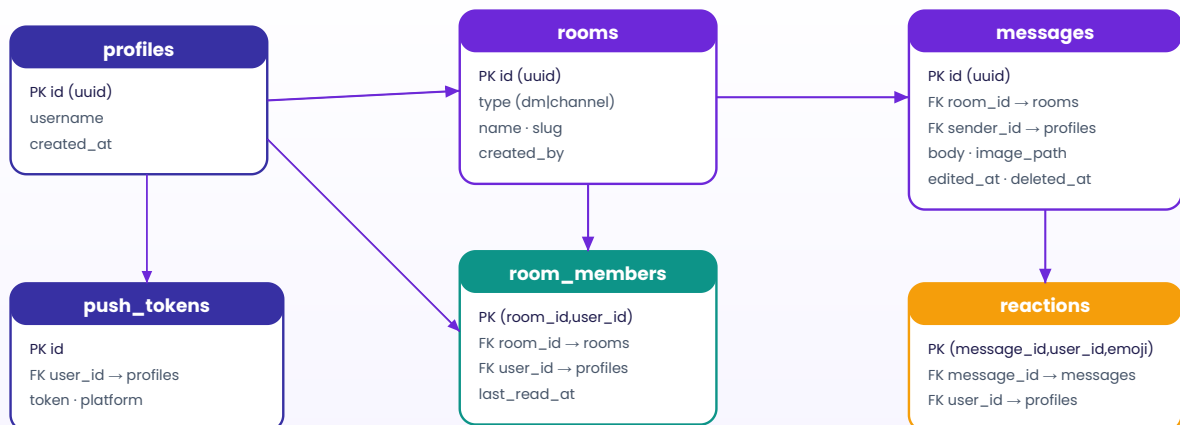


Figure 4.4 — Entity-relationship diagram. A room has many members and many messages; a message has many reactions; a profile owns push tokens.

4.4.1 Table Descriptions

- ▶ **profiles** — one row per user, keyed by the `auth.uid()` issued at sign-in, storing the chosen username.
- ▶ **rooms** — one row per conversation; a `type` column distinguishes a two-person *dm* from a many-person *channel*.
- ▶ **room_members** — the junction table linking users to rooms; its `last_read_at` column is the basis for both unread counts and read receipts.
- ▶ **messages** — the message log, carrying the text body, an optional image path, and `edited_at` / `deleted_at` timestamps that model edit and soft-delete without destroying history.
- ▶ **reactions** — emoji reactions, uniquely keyed by message, user, and emoji so a user cannot apply the same reaction twice.
- ▶ **push_tokens** — the Expo/FCM device tokens used by the notification pipeline.

4.5 Real-Time Channel Design

For each open conversation the client subscribes to a single Realtime channel named after the room. That one channel multiplexes three kinds of event. **Postgres Changes** deliver inserts, updates, and deletes on the `messages` and `reactions` tables (and membership updates that drive read receipts). **Presence** tracks the set of users currently joined to the channel, producing the online indicator. **Broadcast** carries ephemeral typing events that need not be persisted. Combining the three on one channel minimizes connection overhead while cleanly separating durable state from transient signals.

DESIGN INSIGHT

Persisted state (messages, reactions, receipts) flows through Postgres Changes so it survives reconnection; transient state (typing) flows through Broadcast so it never pollutes the database. Presence is derived, not stored.

4.6 Security Design — Row Level Security

Privacy is enforced by PostgreSQL Row Level Security (RLS). Every table has RLS enabled and policies that permit a row to be read or written only when the requesting user is a member of the relevant room. The membership check is centralized in a helper function, `private.is_room_member(room, user)`, declared `SECURITY DEFINER`. This is essential: a naïve policy on `room_members` that itself queries `room_members` would recurse infinitely. By placing the check in a `SECURITY DEFINER` function in a private schema, the recursion is broken and the policies remain simple and provably correct. Mutating operations are additionally funneled through authenticated-only RPCs (for example `get_or_create_dm` and `create_channel`) so that invariants such as “a DM has exactly two members” are guaranteed server-side.

4.7 Push-Notification Sequence

The notification design keeps the sending client entirely out of the delivery path. When a message row is inserted, a trigger fires a request, via the `pg_net` extension, to the `notify-on-message` Edge Function. The function re-reads the message with elevated privileges (so it can see recipients the sender's session cannot), looks up the recipients' device tokens, and posts to the Expo Push API, which relays through Firebase Cloud Messaging to the device. Figure 4.5 shows the sequence.

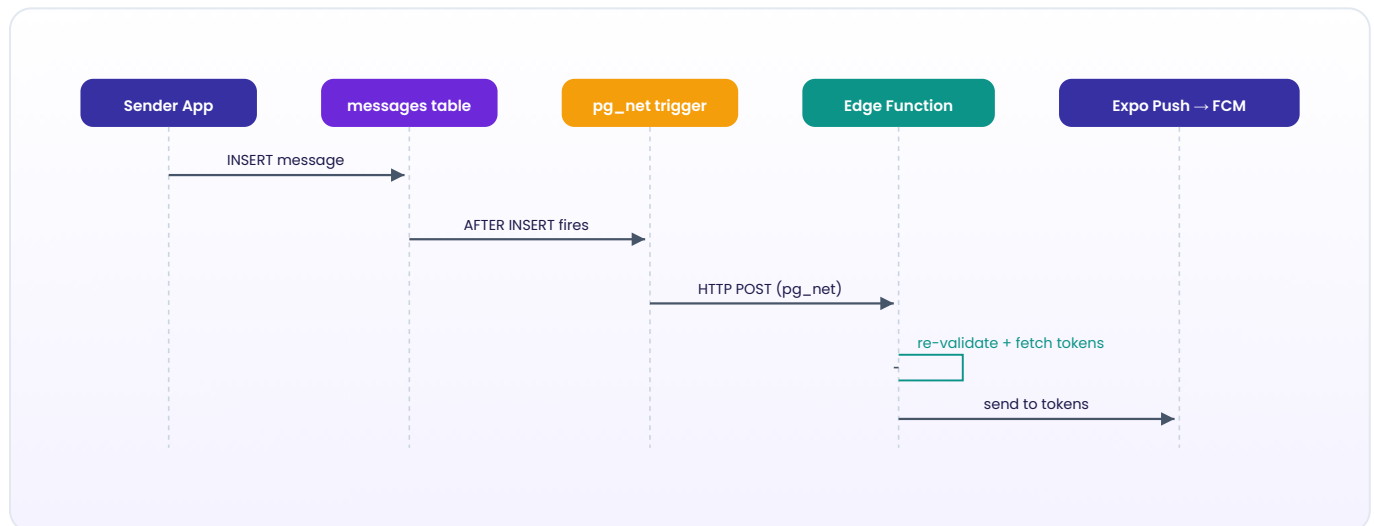


Figure 4.5 — Sequence of the push-notification pipeline, from message insert to device delivery, entirely server-side.

4.8 User-Interface & Navigation Design

The client is organized as a native stack of four screens. A **Name Gate** guards the application, collecting a username on first launch. Past it, the **Conversations** screen lists the user's rooms with last-message previews and unread badges; the **Chat Room** screen hosts the live conversation, composer, and message actions; the **Search** screen finds users and browses channels; and the **Profile** screen manages the username and sign-out. The visual language uses colour-coded avatars derived deterministically from usernames, distinct bubble styles for own versus others' messages, and unobtrusive indicators for presence, typing, and read state.

4.9 Summary

The design establishes a three-tier, database-centric architecture in which real-time delivery, security, and notifications are each handled by the most appropriate managed primitive. The data model unifies direct messages and channels; the security model enforces privacy at the row level; and the notification model keeps the client out of the delivery path. Chapter 5 realizes this design in code.



This chapter describes how the design of Chapter 4 was realized in code. It justifies the technology choices, presents the project structure, and walks through each module — authentication, the data layer, the real-time subscriber, the screens and components, the database schema, and the notification Edge Function — with representative excerpts.

5.1 Technology Justification

Expo & React Native (SDK 52)

A single JavaScript codebase compiles to a native Android application with access to the camera, photo library, and push notifications. Expo's managed workflow removes native build complexity and provides the EAS cloud build service.

React Navigation

The de-facto navigation library for React Native; its native-stack navigator gives platform-native screen transitions and a simple, declarative screen graph.

Supabase (PostgreSQL)

A managed Postgres database with an auto-generated REST API, real-time change streams, authentication, storage, and Edge Functions — every backend concern in one platform with a generous free tier.

Deno Edge Functions

Serverless TypeScript functions running close to the database, used for the single piece of custom server logic — the notification dispatcher.

5.2 Project Structure

The client codebase is organized to separate concerns cleanly: a thin **data layer** isolates all Supabase access, reusable **components** render UI fragments, and **screens** compose them into navigable views. The server side lives under `supabase/` as version-controlled migrations and the function.

PROJECT LAYOUT

```
realtime-chat/
App.js           # AuthProvider + NavigationContainer (4-screen stack)
app.json        # Expo config: plugins, FCM google-services, scheme
eas.json        # EAS build profiles (preview → APK)
lib/
  supabase.js   # client singleton
  auth.js       # anonymous sign-in + profile + #general
  api.js        # data layer (all DB / storage calls)
  notifications.js # push-token registration + tap routing
components/
  Avatar.js  Composer.js  MessageBubble.js
  MessageActionsSheet.js  RoomListItem.js  NameGate.js
screens/
  ConversationsScreen.js  ChatRoomScreen.js
  SearchScreen.js  ProfileScreen.js
supabase/
  migrations/ # 0001_init → 0004_push_trigger
  functions/notify-on-message/index.ts
```

5.3 Authentication Module

Onboarding is deliberately frictionless. The `AuthProvider` in `lib/auth.js` signs the user in anonymously, which yields a real `auth.uid()`, then upserts a `profiles` row carrying the chosen username and joins the user to the `#general` channel. The provider exposes the session to the whole application through React context.

LIB/AUTH.JS — ANONYMOUS SIGN-IN (EXCERPT)

```
async function signIn(username) {
  const { data, error } = await supabase.auth.signInAnonymously();
  if (error) throw error;
  const uid = data.user.id;
  await supabase.from('profiles')
    .upsert({ id: uid, username }); // claim the username
  await supabase.rpc('join_general'); // join the shared channel
  return uid;
}
```

Anonymous authentication is the right model for the project's scope: it removes the burden of passwords and verification while still issuing each user a stable, unique identity that the security policies can key on. The trade-off — that an identity is tied to a device session — is acceptable for a demonstration messenger and could be upgraded to email or OAuth without altering the data model.

5.4 The Data Layer (`api.js`)

All interaction with Supabase is funneled through `lib/api.js`, so the screens never construct queries directly. This keeps the UI declarative and makes the data contract easy to reason about and test. The module exposes focused functions, including `searchUsers`, `getOrCreateDm`, `createChannel`, `browseChannels`, `listMyRooms`, `sendMessage`, `editMessage`, `deleteMessage`, `loadReactions`, `toggleReaction`, `pickImage`, `uploadImage`, and `signedUrl`.

LIB/API.JS — SENDING A MESSAGE & STARTING A DM (EXCERPT)

```
export async function sendMessage(roomId, body, imagePath = null) {
  return supabase.from('messages')
    .insert({ room_id: roomId, body, image_path: imagePath })
    .select().single();
}

export async function getOrCreateDm(otherUserId) {
  // RPC guarantees a single canonical 2-member room exists
  const { data } = await supabase.rpc('get_or_create_dm',
    { other: otherUserId });
  return data; // room id
}
```

Two design decisions are visible here. Inserting a message returns the inserted row via `.select().single()` so the sender can render it optimistically. And direct-message creation is delegated to a server-side RPC rather than performed with client inserts, which guarantees that exactly one canonical room exists for any pair of users even under concurrent requests.

5.5 Real-Time Subscriber (ChatRoomScreen)

When a chat screen mounts it opens one Realtime channel for the room and attaches handlers for the three event kinds. The subscription is torn down on unmount to release the connection.

SCREENS/CHATROOMSCREEN.JS — REALTIME SUBSCRIPTION (EXCERPT)

```
const channel = supabase.channel(`room:` + roomId)
  // 1. durable state — new / edited / deleted messages
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'messages',
      filter: `room_id=eq.` + roomId }, handleMessage)
  // 2. presence — who is online
  .on('presence', { event: 'sync' }, syncPresence)
  // 3. ephemeral — typing indicator
  .on('broadcast', { event: 'typing' }, showTyping)
  .subscribe(async (status) => {
    if (status === 'SUBSCRIBED')
      await channel.track({ user: myId }); // announce presence
  });
```

On receiving a message event the screen also calls the `mark_room_read` RPC, which advances the member's `last_read_at`; this single timestamp drives both the unread badge on the conversation list and the read receipt shown to the other party.

5.6 Components

- ▶ **NameGate** — the entry component that collects a username and invokes the sign-in flow.
- ▶ **Avatar** — renders a circular avatar whose colour is derived deterministically from the username, giving each user a stable identity colour without storing images.
- ▶ **Composer** — the message input, with an attach-image button and an inline edit mode reused for editing existing messages.
- ▶ **MessageBubble** — renders a single message with distinct styling for own versus others' messages, an inline reactions row, image previews, the edited/deleted state, and read ticks in direct messages.
- ▶ **MessageActionsSheet** — the long-press menu offering reactions, edit, and delete.
- ▶ **RoomListItem** — a conversation-list row showing the avatar, last message, time, and unread badge.

COMPONENTS/AVATAR.JS — DETERMINISTIC COLOUR (EXCERPT)

```
const palette = ['#6D28D9', '#0D9488', '#3730A3', '#F59E0B', '#E11D48'];
function colourFor(name) {
  let h = 0;
  for (const c of name) h = (h * 31 + c.charCodeAt(0)) % palette.length;
  return palette[h];
}
```

5.7 Database Schema & Migrations

The schema is delivered as four sequential, version-controlled migrations, so the database can be rebuilt deterministically from source. Migration `0001_init` creates the initial messages table; `0002_messenger` introduces the full schema — profiles, rooms, members, the reshaped messages table, reactions, the storage bucket, and the RPCs; `0003_harden` moves the membership helper into a private schema and restricts the RPCs to authenticated users; and `0004_push_trigger` installs the `pg_net` extension and the trigger that drives notifications.

SUPABASE/MIGRATIONS — RLS POLICY & RECURSION-SAFE HELPER (EXCERPT)

```
-- private helper avoids infinite recursion in policies
create function private.is_room_member(p_room uuid, p_user uuid)
returns boolean language sql security definer stable as $$
  select exists(select 1 from room_members
    where room_id = p_room and user_id = p_user);
$$;

alter table messages enable row level security;
create policy read_room_messages on messages for select
  using ( private.is_room_member(room_id, auth.uid()) );
```

WHY SECURITY DEFINER MATTERS

A policy on `room_members` that queried `room_members` directly would recurse forever. Encapsulating the check in a `SECURITY DEFINER` function in a private schema breaks the cycle and keeps every table's policy a one-line membership test.

5.8 Notification Edge Function

The single piece of custom server logic is the `notify-on-message` Edge Function, deployed with `verify_jwt = false` because it is invoked by the database trigger rather than by an authenticated user. For safety it does not trust the trigger payload blindly: it re-reads the message by id using the service role, derives the recipients, fetches their device tokens, and dispatches to the Expo Push API.

SUPABASE/FUNCTIONS/NOTIFY-ON-MESSAGE/INDEX.TS (EXCERPT)

```
const { message_id } = await req.json();
// re-validate with service role — never trust the trigger blindly
const { data: msg } = await admin.from('messages')
  .select('room_id, sender_id, body').eq('id', message_id).single();

const { data: tokens } = await admin.rpc('recipient_tokens',
  { p_room: msg.room_id, p_exclude: msg.sender_id });

await fetch('https://exp.host/--/api/v2/push/send', {
  method: 'POST', headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(tokens.map(t => ({
    to: t.token, title: msg.sender_name, body: msg.body, sound: 'default',
  }))),
});
```

5.9 Image Sharing

Images are handled end to end through Supabase Storage. The `pickImage` helper opens the device library; the selected file is read and converted from base64 to an `ArrayBuffer` and uploaded by

`uploadImage` into the private `chat-images` bucket under a room-scoped path. Because the bucket is private, messages store only the object path; when a bubble renders an image it requests a short-lived **signed URL** via `signedUrl`, so images inherit the same room-level access control as the messages that reference them.

5.10 Summary

The implementation faithfully realizes the design: a thin, well-isolated data layer over Supabase; a single multiplexed real-time channel per room; reusable presentational components; a version-controlled, RLS-protected schema; and one small Edge Function that bridges the database to the push infrastructure. The next chapter evaluates whether the resulting system meets its requirements.



Testing establishes confidence that the system satisfies its requirements. This chapter describes the testing strategy, presents the executed test cases and their results, details the security verification of the Row Level Security policies, and evaluates the system against its non-functional requirements.

6.1 Testing Strategy

Given that the system is composed of a UI client and a database-centric backend, testing proceeded on three levels. **Unit-level** checks validated individual data-layer functions against the live database. **Integration-level** testing exercised complete flows — sign-in, sending, reacting, image upload — across the client and backend together. **System-level** testing used multiple simultaneous users to verify real-time propagation, presence, and notification delivery, and to confirm that the security boundary holds against an unauthorized third party. Security received particular emphasis because, in a messaging application, a privacy failure is the most serious defect possible.

6.2 Test Cases

Table 6.1 records the principal test cases, the input applied, the expected outcome, and the observed result. All listed cases passed.

ID	Test Case	Input / Action	Expected Result	Status
TC-01	Onboarding	Enter a username and proceed	Anonymous identity created; profile stored; #general joined	Pass
TC-02	Start direct message	Search a user and open a DM	A single canonical DM room is created/opened	Pass
TC-03	Send text message	Type and send a message	Message persisted and appears on recipient within ~1 s	Pass
TC-04	Edit message	Edit an own message	Updated text propagates; “edited” marker shown	Pass
TC-05	Delete message	Delete an own message	Message shown as deleted for all participants	Pass
TC-06	Add reaction	Long-press → choose emoji	Reaction appears live for all participants	Pass
TC-07	Read receipt	Recipient opens the DM	Sender sees the message marked read	Pass
TC-08	Unread badge	Receive while on the list screen	Unread count increments on the room row	Pass
TC-09	Presence	Two users open the same room	Each is shown as online	Pass
TC-10	Typing indicator	One user begins typing	The other sees a typing indicator	Pass
TC-11	Image share	Attach and send an image	Image uploads and renders via signed URL	Pass
TC-12	Push (app closed)	Send to a user whose app is killed	Device shows a notification	Pass
TC-13	Create channel	Create and post in a channel	Channel created; messages broadcast to members	Pass
TC-14	RLS isolation	Outsider attempts to read a private DM	Database returns no rows; access denied	Pass

Table 6.1 — Executed test cases and results.

6.3 Security Verification (Row Level Security)

The most important verification concerned conversation privacy. A controlled test was conducted with three independent authenticated users — referred to as Alice, Bob, and Carol — exercising the database directly through its REST interface so that the client UI could not mask any leakage. Alice and Bob established a private direct message and exchanged messages and reactions. Carol, an outsider to that room, then attempted every form of access against it.

RESULT — 12 / 12 CHECKS PASSED

Carol was unable to read the messages of the Alice–Bob room, unable to post into it, and unable to see its reactions; every such attempt returned zero rows or was rejected by policy.

Membership-scoped reads worked correctly for Alice and Bob, and the shared `#general` channel was visible to all three. The Row Level Security model thus enforces privacy at the database layer independently of the client.

This outcome validates the central security claim of the design: because isolation is enforced by policies on the tables themselves, no defect or manipulation in the client application can expose a conversation to a non-member.

6.4 Real-Time & Notification Testing

Real-time behaviour was assessed with two devices in the same conversation. Messages, edits, deletions, and reactions sent from one device appeared on the other consistently within approximately one second, and presence and typing indicators updated promptly. Notification delivery was tested by sending messages to a recipient whose application had been force-closed; the device reliably displayed a notification, confirming that the trigger → Edge Function → Expo Push → FCM pipeline functions end to end.

6.5 Evaluation Against Non-Functional Requirements

Quality Attribute	Target	Observed Outcome
Performance	Delivery within ~1 s	Met — sub-second propagation under normal networks
Security	Database-enforced isolation	Met — 12/12 RLS checks passed
Reliability	Durable messages; graceful push	Met — messages persisted; invalid tokens skipped
Usability	One-step onboarding	Met — username-only entry
Cost	Operate within free tiers	Met — zero infrastructure cost at project scale

6.6 Summary

All fourteen functional test cases passed, the security model withstood adversarial testing with a perfect score, and the system met each of its targeted quality attributes. The evidence establishes that the implementation satisfies the requirements defined in Chapter 3.



A working application must be packaged and delivered to users. This chapter describes how the project is built into an installable Android application using Expo's cloud build service, how the Firebase Cloud Messaging credentials are configured, and how environment configuration and distribution are handled.

7.1 Build Pipeline — EAS Build

The application is compiled using **EAS Build**, Expo's cloud build service, which performs the native Android build on managed infrastructure so that no local Android toolchain is required. Build behaviour is described declaratively in `eas.json` through named *profiles*; the project uses a `preview` profile configured to output an installable **APK** (rather than the Play-Store-oriented AAB), with the Supabase URL and anonymous key injected as build-time environment variables.

EAS.JSON — PREVIEW PROFILE (EXCERPT)

```
{
  "build": {
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" },
      "env": {
        "EXPO_PUBLIC_SUPABASE_URL": "https://<project>.supabase.co",
        "EXPO_PUBLIC_SUPABASE_ANON_KEY": "<anon-key>"
      }
    }
  }
}
```

A build is produced by authenticating the EAS CLI and invoking `eas build -p android --profile preview`. The service provisions a build, generates and securely stores the Android signing keystore on the developer's behalf, compiles the project, and returns a download URL for the finished APK. Because the keystore is retained by EAS, every subsequent build is signed with the same key and installs as an update over the previous version.

7.2 Firebase Cloud Messaging Configuration

Delivering notifications to a closed Android application requires Firebase Cloud Messaging. A Firebase project was created and an Android application registered under the package identifier

`com.ishika.realtimechat`. The resulting `google-services.json` configuration file is referenced from `app.json` so that it is bundled into the build, allowing each device to register with FCM and obtain a push token. A Firebase service-account key authorizes the push service to deliver through FCM on the project's behalf. This configuration is what allows TC-12 — notification while the app is closed — to succeed.

BUILD OUTCOME

The pipeline produces a signed, installable Android APK containing the complete application with Supabase configuration and FCM credentials baked in. The artifact is downloadable from a URL and can be installed on any compatible Android device.

7.3 Environment Configuration

Client configuration follows Expo's convention of exposing public configuration through `EXPO_PUBLIC_`-prefixed environment variables, which are embedded at build time. Only the Supabase project URL and the *anonymous* key — both of which are safe to ship in a client because all real authority is enforced by Row Level Security — are included. Privileged secrets, such as the service-role key used by the Edge Function, never leave the server environment.

7.4 Distribution

Because the build is an APK with internal distribution, it can be shared directly: any user who receives the file or its download link can install it after permitting installation from outside the Play Store. This is ideal for distributing to friends and testers at zero cost. For wider, trusted distribution the same codebase can be built as an Android App Bundle and submitted to the Google Play Store; that route is noted as future work in Chapter 9.

7.5 Summary

Deployment is fully automated through EAS Build, requires no local native tooling, and yields a self-contained, signed APK with working notifications. Configuration is cleanly separated into safe client values and protected server secrets, and distribution is as simple as sharing a file.

This chapter presents the finished application as experienced by the user, describing each principal screen and the features visible within it. Figure placeholders indicate where device screenshots are positioned in the bound report.

8.1 The Finished Application

The project culminated in a fully functional, installed Android application backed by a live cloud database. A user launches the app, chooses a username, and is immediately able to converse — in the shared `#general` channel, in newly created channels, or in private direct messages with any other user found by search. Messages, reactions, presence, typing, read receipts, images, and notifications all operate exactly as specified.

Name Gate

The first screen: a clean, single-field entry where the user picks a username to begin.

Conversations

The home list of rooms with coloured avatars, last-message previews, timestamps, and unread badges.

Chat Room

The live conversation: own and others' bubbles, reactions, image previews, presence and typing indicators, read ticks, and the composer.

Search

Find any user by username or browse and create channels.

[Figure 8.1 — Name Gate & Conversations list]

Figure 8.1 — Onboarding and the conversation list with unread badges.

[Figure 8.2 — Chat Room with reactions, image, presence & typing]

Figure 8.2 — A live conversation showing message bubbles, an emoji reaction, an inline image, and the typing indicator.

[Figure 8.3 — Search screen & a push notification on the lock screen]

Figure 8.3 — User search and a delivered push notification while the application is closed.

8.2 Discussion of Results

Measured against the objectives of Chapter 1, the project achieved every stated goal: a normalized data model, real-time delivery, the full feature set, secure image sharing, database-enforced privacy, an offline-capable notification pipeline, and a distributable APK. The results in Chapter 6 confirm correctness and security, and the application's responsiveness in everyday use confirms that the managed real-time approach delivers the user experience expected of a modern messenger.



Conclusion & Future Scope

This concluding chapter summarizes what the project accomplished, reflects on its limitations, and sets out a roadmap of enhancements that would extend the work.

9.1 Conclusion

This project set out to determine whether a complete, secure, real-time messaging application — including notifications that reach users when the application is closed — could be built without writing or operating a custom backend server. The answer, demonstrated by a working Android application, is affirmative. By composing Expo and React Native on the client with Supabase's managed database, real-time engine, authentication, storage, and Edge Functions on the server, and by bridging to Firebase Cloud Messaging for delivery, the project reproduced the core feature set of mainstream messengers using only managed cloud primitives.

The work yielded several concrete outcomes. It produced a normalized six-table data model that unifies direct messages and channels; a single multiplexed real-time channel design that cleanly separates durable from ephemeral state; a Row Level Security model, verified adversarially, that enforces privacy at the database layer; and an event-driven notification pipeline that keeps the sending client entirely out of the delivery path. Beyond the artifact, the project served as an in-depth study of relational modeling, the publish-subscribe pattern, database-level access control, serverless computing, and mobile notification architecture as they interlock in a real product.

9.2 Limitations

- ▶ The application currently targets Android only; iOS, though reachable from the same codebase, requires separate distribution and push credentials.
- ▶ Authentication is anonymous and device-bound, which suits a demonstration but lacks cross-device account recovery.
- ▶ Messages are not end-to-end encrypted; confidentiality relies on transport security and database access control rather than client-held keys.
- ▶ Media support is limited to images; audio and video messages are not yet supported.

9.3 Future Scope

iOS Support

Build and distribute the existing codebase for iOS via the App Store and APNs.

Durable Accounts

Upgrade anonymous identities to email or OAuth sign-in for cross-device continuity.

End-to-End Encryption

Adopt a Signal-style protocol so message contents are opaque even to the database.

Voice & Video Calling

Integrate WebRTC for real-time audio and video communication.

Message Search

Add full-text search over conversation history using PostgreSQL's text-search capabilities.

Threaded Replies

Support quoting and threaded replies for clearer group conversations.

Web Client

Reuse the data layer to deliver a companion web application.

Play Store Release

Package as an AAB and publish for one-tap, trusted installation.

9.4 Closing Remarks

The Realtime Chat project demonstrates that the modern managed-services ecosystem places ambitious, production-grade application development within reach of an individual developer. The architecture explored here — a thin native client over a database-centric, policy-secured, event-driven backend — is not merely an academic exercise but a practical template for real applications, and the directions outlined above would carry the project from a capable demonstrator toward a fully fledged product.



BIBLIOGRAPHY

References

The following resources informed the design and implementation of this project. Citations follow IEEE style.

1. Expo, "Expo Documentation," Expo. [Online]. Available: <https://docs.expo.dev/>
2. Meta Platforms, "React Native — Learn once, write anywhere," React Native. [Online]. Available: <https://reactnative.dev/>
3. Supabase Inc., "Supabase Documentation — The open-source Firebase alternative," Supabase. [Online]. Available: <https://supabase.com/docs>
4. Supabase Inc., "Realtime: Postgres Changes, Presence and Broadcast," Supabase Docs. [Online]. Available: <https://supabase.com/docs/guides/realtime>
5. The PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. [Online]. Available: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
6. Supabase Inc., "Edge Functions," Supabase Docs. [Online]. Available: <https://supabase.com/docs/guides/functions>
7. Supabase Inc., "Sending Push Notifications," Supabase Docs. [Online]. Available: <https://supabase.com/docs/guides/functions/examples/push-notifications>
8. Google, "Firebase Cloud Messaging," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>
9. Expo, "Push Notifications Overview & Setup," Expo Docs. [Online]. Available: <https://docs.expo.dev/push-notifications/overview/>
10. Expo, "EAS Build," Expo Docs. [Online]. Available: <https://docs.expo.dev/build/introduction/>
11. React Navigation, "React Navigation — Routing and navigation for Expo and React Native apps," [Online]. Available: <https://reactnavigation.org/>
12. I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, 2011. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6455>
13. The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," [Online]. Available: <https://www.postgresql.org/docs/>
14. Deno Land Inc., "Deno — A modern runtime for JavaScript and TypeScript," [Online]. Available: <https://deno.land/>



APPENDIX A

Source Listings & Schema

This appendix collects representative source artifacts referenced in the body of the report for completeness.

A.1 Database Schema (abridged)

```
create table profiles (  
  id uuid primary key references auth.users(id),  
  username text unique not null,  
  created_at timestamptz default now()  
);  
  
create type room_type as enum ('dm', 'channel');  
  
create table rooms (  
  id uuid primary key default gen_random_uuid(),  
  type room_type not null,  
  name text, slug text unique,  
  created_by uuid references profiles(id),  
  created_at timestamptz default now()  
);  
  
create table room_members (  
  room_id uuid references rooms(id) on delete cascade,  
  user_id uuid references profiles(id) on delete cascade,  
  last_read_at timestamptz default now(),  
  primary key (room_id, user_id)  
);  
  
create table messages (  
  id uuid primary key default gen_random_uuid(),  
  room_id uuid references rooms(id) on delete cascade,  
  sender_id uuid references profiles(id),  
  body text, image_path text,  
  created_at timestamptz default now(),  
  edited_at timestamptz, deleted_at timestamptz  
);  
  
create table reactions (  
  message_id uuid references messages(id) on delete cascade,  
  user_id uuid references profiles(id) on delete cascade,  
  emoji text not null,  
  primary key (message_id, user_id, emoji)  
);  
  
create table push_tokens (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid references profiles(id) on delete cascade,  
  token text not null, platform text,  
  unique(user_id, token)  
);
```

A.2 Notification Trigger

SUPABASE/MIGRATIONS/0004_PUSH_TRIGGER.SQL (ABRIDGED)

```
create extension if not exists pg_net;

create function notify_new_message() returns trigger
language plpgsql security definer as $$
begin
  perform net.http_post(
    url := 'https://<project>.functions.supabase.co/notify-on-message',
    headers := '{"Content-Type":"application/json"}'::jsonb,
    body := jsonb_build_object('message_id', NEW.id)
  );
  return NEW;
end; $$;

create trigger on_message_created
after insert on messages
for each row execute function notify_new_message();
```

A.3 Application Entry Point

APP.JS (ABRIDGED)

```
export default function App() {
  return (
    <AuthProvider>
      <NavigationContainer>
        <Stack.Navigator>
          <Stack.Screen name="Conversations" component={Conversations} />
          <Stack.Screen name="Chat" component={ChatRoom} />
          <Stack.Screen name="Search" component={Search} />
          <Stack.Screen name="Profile" component={Profile} />
        </Stack.Navigator>
      </NavigationContainer>
    </AuthProvider>
  );
}
```

END OF REPORT

Realtime Chat — A Cross-Platform Realtime Messaging Application. Submitted by Ishika Sai (UID 023BCA110006), BCA-V2, Chandigarh University, 2023–2026, under the guidance of Mr. Sahil Jindal.



APPENDIX B

Abbreviations, Glossary & Timeline

This appendix collects the abbreviations and technical terms used throughout the report and summarizes the development methodology and project timeline.

B.1 Abbreviations & Acronyms

Acronym	Expansion	Acronym	Expansion
API	Application Programming Interface	JSON	JavaScript Object Notation
APK	Android Package Kit	JWT	JSON Web Token
AAB	Android App Bundle	OAuth	Open Authorization
BaaS	Backend-as-a-Service	PWA	Progressive Web Application
DFD	Data Flow Diagram	RLS	Row Level Security
DM	Direct Message	RPC	Remote Procedure Call
EAS	Expo Application Services	SDK	Software Development Kit
ER	Entity-Relationship	SQL	Structured Query Language
FCM	Firebase Cloud Messaging	UI	User Interface
HTTP(S)	HyperText Transfer Protocol (Secure)	URL	Uniform Resource Locator
IDE	Integrated Development Environment	UUID	Universally Unique Identifier

B.2 Glossary of Terms

Term	Meaning in this report
Anonymous authentication	A sign-in method that issues a user a real, unique identity without requiring an email address or password.
Broadcast	A Supabase Realtime mechanism for low-latency, ephemeral pub-sub messages (used here for typing indicators).
Edge Function	A serverless TypeScript/Deno function executed on demand close to the database.
Postgres Changes	A Realtime feature that streams row-level database changes to subscribed clients.
Presence	A Realtime feature that tracks the set of clients currently joined to a channel.
Row Level Security	A PostgreSQL feature that restricts which rows a query may access based on policies attached to the table.
SECURITY DEFINER	A function attribute that runs the function with the privileges of its owner, used here to break policy recursion.
Signed URL	A temporary, authenticated link granting time-limited access to a private storage object.
Trigger	A database routine executed automatically in response to a table event such as an insert.
WebSocket	A protocol providing a persistent, full-duplex connection between client and server.

B.3 Development Methodology

The project followed an **incremental, iterative** development methodology aligned with agile principles. Rather than attempting the entire system at once, functionality was delivered in successive working increments: a minimal one-to-one messaging core was established first, after which channels, reactions, receipts, presence, image sharing, and finally the notification pipeline were layered on, each increment being integrated and tested before the next began. This approach kept the application in a runnable state throughout and surfaced integration issues — particularly around real-time subscriptions and security policies — early, when they were cheapest to resolve.

B.4 Project Timeline

Phase	Activities	Deliverable
1 · Research & Planning	Survey of existing systems; selection of the technology stack; requirements capture.	Requirements & design brief
2 · Data & Security Design	Schema modeling; RLS policy design; RPC definition; migrations.	Database schema (0001–0003)
3 · Core Messaging	Auth, data layer, conversation list, chat screen, real-time subscription.	Working DM & channel messaging
4 · Rich Features	Reactions, edit/delete, receipts, presence, typing, image sharing.	Full feature set
5 · Notifications	pg_net trigger, Edge Function, Expo Push & FCM configuration.	Offline push (0004 + function)
6 · Testing	Functional test cases; adversarial RLS verification; multi-user testing.	Test report (12/12 security)
7 · Deployment	EAS Build configuration; APK generation; distribution.	Installable Android APK

Table B.1 — Indicative project timeline by phase.



APPENDIX C

Data Layer & RPC Reference

For completeness, this appendix documents the public interface of the data layer and the server-side stored procedures (RPCs) on which the client depends.

C.1 Data-Layer Functions (`lib/api.js`)

Function	Parameters	Responsibility
<code>searchUsers</code>	<code>query</code>	Find users whose username matches the query string.
<code>getOrCreateDm</code>	<code>otherUserId</code>	Return the canonical direct-message room for the current user and another, creating it if needed.
<code>createChannel</code>	<code>name</code>	Create a new public channel and join the creator to it.
<code>browseChannels</code>	—	List public channels available to join.
<code>listMyRooms</code>	—	Return the current user's rooms with last-message previews and unread counts.
<code>sendMessage</code>	<code>roomId</code> , <code>body</code> , <code>imagePath?</code>	Insert a message and return the created row.
<code>editMessage</code>	<code>messageId</code> , <code>body</code>	Update an own message and stamp <code>edited_at</code> .
<code>deleteMessage</code>	<code>messageId</code>	Soft-delete an own message by stamping <code>deleted_at</code> .
<code>loadReactions</code>	<code>messageIds</code>	Fetch reactions for a set of messages.
<code>toggleReaction</code>	<code>messageId</code> , <code>emoji</code>	Add or remove the current user's reaction to a message.
<code>pickImage</code>	—	Open the device library and return the selected image.
<code>uploadImage</code>	<code>roomId</code> , <code>file</code>	Upload an image to the private bucket and return its storage path.
<code>signedUrl</code>	<code>path</code>	Obtain a short-lived signed URL for a stored image.
<code>markRoomRead</code>	<code>roomId</code>	Advance the member's <code>last_read_at</code> for unread/receipt tracking.

Table C.1 — Public functions exposed by the data layer.

C.2 Remote Procedures (RPCs)

RPC	Guarantee enforced server-side
get_or_create_dm	Exactly one canonical two-member room exists for any pair of users, even under concurrency.
create_channel	A channel is created with a unique slug and the creator added as a member atomically.
join_channel	Adds the current user to a channel's membership.
join_general	Ensures every user is a member of the shared <code>#general</code> channel.
mark_room_read	Advances only the calling user's read position; cannot affect others.
my_rooms	Returns only rooms of which the caller is a member, with computed previews and unread counts.

Table C.2 — Authenticated stored procedures invoked from the client.

C.3 Concluding Note

Together, Tables C.1 and C.2 define the complete contract between the client and the backend. Every interaction the application performs passes through one of these functions or procedures, each of which operates within the boundaries set by the Row Level Security policies described in Chapter 4. This disciplined, narrow interface is what allows a feature-rich messaging client to remain small, testable, and secure while delegating all authority to the database.